SP-201 Red Multi-Agent Research & Fact-Checking Pipeline

SOFTWARE REQUIREMENTS SPECIFICATION (SRS)

Senior Project Section 2

Spring 2026

February 15, 2026

Professor Perry

			
Melike Ozcelik Team Leader Developer	Raaid Khan Developer Documentation	Nha Phan Developer	Afra Kurudirek Documentation

1.0 Introduction

1.1 Overview

The design and implementation of a multi-agent autonomous research and fact-checking system using existing big language models and agent orchestration frameworks is the main goal of this project. In order to generate a verified, credited research report, the system is built to process complicated, open-ended research prompts by breaking them down into structured subtasks and coordinating several specialized agents.

A central Manager Agent evaluates the user's prompt and assigns tasks to several worker agents in the architecture Manager–Worker pattern. A search agent uses free, public APIs available to retrieve pertinent data from other data sources. A Fact-Checker Agent cross-references assertions with the original sources to identify discrepancies and lessen hallucinations, while a Synthesizer Agent incorporates the collected data into a logical draft.

1.2 Project Scope

This project's scope includes automatic fact-checking of created content, structured prompt deconstruction, multi-agent task orchestration, session state management, and external data

retrieval via publically accessible APIs.

The system will show how coordinated autonomous agents can raise the caliber and legitimacy of research results produced by AI.

The project lacks enterprise-grade scaling capabilities, paid API integration, and large-scale deployment.

1.3 Definitions and Acronyms

- Agent: A software entity that operates independently to accomplish a given task.
- Manager Agent: Responsible for assigning tasks to worker agents.
- Search Agent: Acquires data from external resources
- Synthesizer Agent: Develops a well-structured research draft
- Fact-Checker Agent: Validates claims with sources found
- LLM: Large Language Model
- ADK: Agent Development Kit

1.4 Assumptions

- Prompts will be given by users in English, which is their natural language.
- Throughout development, public, free-tier APIs (such as search and LLM APIs) will continue to be accessible.
- Users are technically literate enough to use a simple web interface or CLI.
- Throughout system operation, a reliable internet connection is accessible.

1.5 References

- Google ADK Documentation
- Google AI Studio Documentation

2.0 Constraints

2.1 Latency and Timeouts

Long-running multi-agent workflows without premature HTTP timeouts must be supported by the system.

To handle network instability, rate limiting, and transient API failures, the system will have structured API error handling and retry logic.

2.2 Token Limits

To guarantee compatibility with LLM context window requirements, the system will impose prompt size limits.

Prompts that over predetermined token thresholds will be truncated, summarized, or rejected by the Manager Agent.

3.0 Functional Requirements

3.1 Manager Agent – The Orchestrator

The Manager Agent will:

- Accept a study prompt that was supplied by a user.
- Break down the prompt into at least three well-structured subquestions.
- Assign one or more instances of the Search Agent to each sub-question.
- Organize the flow of the workflow for each agent.
- If the Fact-Checker Agent rejects the manuscript, start the search or synthesis over.
- Return the final, approved output to the user after aggregating it.

3.2 Search Agent

The search agent will:

- Access external data sources through authorized public APIs.
- Obtain pertinent textual information about the designated sub-questions.
- Provide organized search results with URLs and source titles.
- Utilize specified retry mechanisms to address API errors.

3.3 Synthesizer Agent

- The synthesizer agent will:
- Combine search results into a draft that is logically organized and cohesive.
- Keep the citation references for the sources you have retrieved.
- Create output that can be seen in the user interface in markdown format.

3.4 Fact-Checker Agent

The Fact checking agent will:

- Compare the synthesized statements with the retrieved original content.
- Determine which allegations are unsubstantiated or unverifiable.
- Flag or reject drafts that are inconsistent.
- When required, initiate the re-execution of the Search or Synthesis phases.

3.5 Session and State Management

The system will:

- Throughout the execution of many agents, preserve the session state.
- Follow the pending, decomposing, searching, synthesizing, fact-checking, and finalizing steps of the procedure.
- Keep track of intermediate outputs for debugging and traceability.

4.0 Non-Functional Requirements

4.1 Security

No permanently identifiable user data will be stored by the system.
Environment variables must be used to safely store API keys and credentials.

4.2 Capacity

During development testing, the system must be able to support a minimum of one concurrent active session. In order to facilitate future multi-user scaling, the architecture must be flexible.

4.3 Usability

The system will offer a basic web-based interface or CLI.
Real-time workflow status updates will be shown on the interface.
Clickable citations must be included in the finished product.

4.4 Other

All factual statements in the final report must have verifiable citations provided by the system.
Through visible state transitions, the workflow will be auditable.

5.0 External Interface Requirements

5.1 User Interface Requirements

The system will:

- An input field that is prompt
- A status indicator that shows agent activity in real time
- An output section that is structured and shows markdown material
- Links that can be clicked to access citations

5.2 APIs

The system will be integrated with:

- Data retrieval using the DuckDuckGo API (or a comparable public search API)
- Gemini API for analyzing huge language models
- Optional MySQL database storage for search results

5.3 Database

All contact using the API must take place over HTTPS.
Communication protocols based on REST must be supported by the system.

6.0 Analysis

6.1 State Transition Diagrams

The following order will be followed by the system workflow:
Pending → Breaking down → Looking → Combining → Verifying → Completing.
The procedure will return to searching or synthesizing if validation is unsuccessful.

6.2 Data Flow

Sequence of data flow:
External APIs → Manager Agent → Synthesizer Agent → Fact-Checker Agent → Manager Agent → User → Manager Agent → Search Agent.